

Manual Proyecto 4 - K means

Hola profe, como sabe; debe guardar el archivo llamado “bird_small.txt” en el mismo directorio que el archivo “Py4_A010201943.py”.

Primero, la función “**load_image_data()**” se encarga de leer el archivo “**bird_small.txt**”. Primero interpreta la línea de encabezado para obtener las dimensiones de la imagen, Luego, lee el resto de las líneas, que son los valores de intensidad de cada pixel, y los convierte en una matriz NumPy. Finalmente, reorganiza y normaliza estos datos para crear una lista de píxeles X (matriz de N x 3), donde cada fila representa el color (R, G, B) de un píxel.

```
"""
Leer archivo de datos de imagen
"""
def load_image_data():
    with open("bird_small.txt", "r") as file:
        # Para encabezado:
        dims = list(map(int, file.readline().strip().split()))

        # Vector con todos los valores de los pixeles
        pixel_values = [int(line.strip()) for line in file if line.strip()]

    data = np.array(pixel_values, dtype=np.uint8)
    # Remodelar los datos a la forma original de la imagen
    image_resized = data.reshape(dims[2], dims[0], dims[1]).transpose(1, 2, 0)
    # Normalizar los datos a un rango de 0 a 1 para el procesamiento
    image_normalized = image_resized / 255.0

    X = image_normalized.reshape(-1, 3) # Cada fila es un píxel (R, G, B)

    return X, image_normalized
```

La función “**kMeansInitCentroids(X, K)**” prepara el algoritmo para empezar. Su propósito es seleccionar “**K**” centroides iniciales de forma aleatoria directamente del conjunto de datos “**X**”. Esto se logra reordenando los índices de los píxeles y eligiendo los primeros “**K**” como los puntos de partida para los clusters de colores.

```

"""
Inicializa K centroides seleccionando K puntos aleatorios de X.
"""
def kMeansInitCentroids(X, K):
    m, n = X.shape
    # Genera K índices aleatorios sin repetición del rango de filas de X
    random_indices = np.random.permutation(m)
    # Selecciona los primeros K puntos como centroides
    centroids = X[random_indices[:K]]

    return centroids

```

La función “**computeCentroids(X, idx, K)**” ejecuta el segundo paso del algoritmo: mover los centroides. Agrupa todos los píxeles según las asignaciones del paso anterior. Para cada grupo (o cluster), calcula el color promedio de todos los píxeles que lo componen. Este color promedio se convierte en la nueva posición del centroide para la siguiente iteración.

```

"""
Asigna cada dato a su centroide más cercano.
"""
def findClosestCentroids(X, centroids):

    K = centroids.shape[0]
    m = X.shape[0]
    idx = np.zeros(m, dtype = int)

    for i in range(m):
        # Calcular la distancia de un punto a TODOS los centroides a la vez
        distances = np.linalg.norm(X[i] - centroids, axis=1)
        # Encontrar el índice del centroide con la menor distancia
        idx[i] = np.argmin(distances)

    return idx

```

La función “**runkMeans(X, initial_centroids, max_iters)**” es la que controla todo el proceso. Llama en un bucle a “**findClosestCentroids**” y “**computeCentroids**” un número determinado de veces según las “max_iters”. En cada iteración, las asignaciones de los píxeles y los centroides de color se refinan, acercándose a una solución óptima.

```

"""
Ejecuta el algoritmo K-means completo.
"""
def runkMeans(X, initial_centroids, max_iters):
    K = initial_centroids.shape[0]
    centroids = initial_centroids

    for i in range(max_iters):
        print(f"Iteración de K-means {i + 1}/{max_iters}...")

        # Asignar cada punto a su centroide más cercano
        idx = findClosestCentroids(X, centroids)

        # Recalcular los centroides
        centroids = computeCentroids(X, idx, K)

    return centroids, idx

```

Por último, en la sección “main” es en la que se llama cada función anteriormente mencionada, y se establecen los parámetros principales del algoritmo k-means, definí $k = 16$ y $\text{max_iters} = 10$. Por otro lado, primero se llama la función `load_image_data`, luego se inicializan los primeros 16 centroides de manera aleatoria con `kMeansInitCentroids`, luego se ejecuta `runkMeans` para encontrar los 16 ‘colores’ finales para representar la imagen, por último se visualiza la imagen original y la reconstruida con la imagen comprimida.